

# Chapter Name

## Data Handling

### Index:

1. Data Types
2. Mutable and Immutable Types
3. Operators
4. Expressions
5. Introduction to Python Standard Library Modules
6. Debugging

## 1. Data Types

Data types is the way of representing the type of any data

### Python offers following built-in core data types:

1. Numbers
2. String
3. List
4. Tuple
5. Sets
6. Dictionary

### 1. Number:

- a. Integer
- b. Floating Point Numbers
- c. Complex Number

### c. Complex Number

1. Python represents complex numbers as pair of floating point numbers
2.  $a+bi$  or  $a+bj$

In [ ]:

```
1 a=0.5+3.1j
2 b=1.5+2j
3 print(a,b)
```

In [ ]:

```
1 print(s1[-7])
```

## Immutable Nature of String

In [ ]:

```
1 s="Python"
2 print(s)
3 s[0]='p'
4 print(s)
```

## 3. List

1. A List in Python represents a list of comma-separated values of any datatype between square brackets[].
2. In List each elements can be individually accessed using its INDEX .
3. This index can be +ve or -ve
4. +ve index -> 0,1,2.....Size-1
5. -ve index -> -1,-2,-3.....-Size
6. List is mutable type in nature

### Format of Creating List Variable:

- Variablename = [value1,value2,value3,.....N]

In [ ]:

```
1 l1=[1,2,3,4,5]
2 l2=[1.5,2.5,3.5,4.5,5.5]
3 l3=['a','b','c','d','e']
4 l4=['aman','sara','naman','risabh']
5 l5=[1,2,1.5,2.5,'a','b','aman','sara']
6 print(l1)
7 print(l2)
8 print(l3)
9 print(l4)
10 print(l5)
11 print(type(l1))
```

### How to access any element from List

In [ ]:

```
1 l5=[1,2,1.5,2.5,'a','b','aman','sara']
2 print(l5,len(l5))
3 print(l5[7],l5[-1])
4 print(l5[3],l5[-5])
```

In [ ]:

```

1  l5=[1,2,1.5,2.5,'a','b','aman','sara']
2  print(l5)
3  l5[0]=100
4  print(l5)
5  l5[-1]="dhawal"
6  print(l5)

```

In [ ]:

```

1  l5=[1,2,1.5,2.5,'a','b','aman','sara']
2  print(l5[-1][0])
3  print(l5[-2][-1])

```

## 4.tuples

1. A Tuple in Python represents a list of comma-separated values of any datatype between function brackets().
2. In Tuple each elements can be individually accessed using its INDEX .
3. This index can be +ve or -ve
4. +ve index -> 0,1,2.....Size-1
5. -ve index -> -1,-2,-3.....-Size
6. Tuple is immutable type in nature

### Format of Creating List Variable:

- Variablename = (value1,value2,value3,.....N)

In [ ]:

```

1  l1=(1,2,3,4,5)
2  l2=(1.5,2.5,3.5,4.5,5.5)
3  l3=('a','b','c','d','e')
4  l4=('aman','sara','naman','risabh')
5  l5=(1,2,1.5,2.5,'a','b','aman','sara')
6  print(l1)
7  print(l2)
8  print(l3)
9  print(l4)
10 print(l5)
11 print(type(l1))

```

In [ ]:

```

1  l1=(1,2,3,4,5)
2  print(l1)
3  l1[0]=100
4  print(l1)

```

## 5. Sets

1. Sets are created by specifying comma separated values enclosed in curly brackets {}
2. Sets elements are unordered and unindexed unlike lists.
3. Sets do not allow duplicate entries elements.
4. Sets cannot contain mutable elements.
5. Sets is mutable type in nature

**Format of Creating Sets Variable:**

- Variablename = {value1,value2,value3,.....N}

In [ ]:

```
1 s={1,2,3,4,3,4,3,3,3}
2 print(s,type(s))
```

In [ ]:

```
1 print(s[0])
```

In [ ]:

```
1 s={1,2,3.5,6.5,'aman',[10,20,30]}
2 print(s)
```

In [ ]:

```
1 s={1,2,3,4,'aman',(10,20),"[1,2]"}
2 print(s)
```

## 5. Dictionary

1. Dictionary is an unordered set of comma separated key:value pairs within {}
2. Their is no built-in index available
3. Dictionary is mutable in nature(only value)
4. Dictionary keys must be immutable in nature.
5. No two keys can be the same in dictionary.

**Format of creating Dictionary:**

-> Dictionaryvariablename = {key:value,key:value,key:value,.....n}

In [ ]:

```
1 D1={10:'zero',11:'one',12:'two',13:'three'}
2 print(D1,type(D1))
```

In [ ]:

```
1 D2={'zero':0, 'one':1, 'two':2, 'three':3}
2 print(D2,type(D2))
```

In [ ]:

```
1 D3={'rollno':1111, 'class':'X', 'sec':'A', 'Clast':'Aman'}
2 print(D3,type(D3))
```

In [ ]:

```
1 D4={1: 'one', (1,2): "Number"}
2 print(D4,type(D4))
```

## How to access any element of Dictionary

In [ ]:

```
1 print(D1)
2 print(D1[13])
3 print(D1[10])
```

In [ ]:

```
1 D2={'zero':0, 'one':1, 'two':2, 'three':3}
2 print(D2)
3 print(D2['zero'])
```

In [ ]:

```
1 D3={'rollno':1111, 'class':'X', 'sec':'A', 'Clast':'Aman', 'rollno':2222}
2 print(D3)
3 print(D3['rollno'])
```

In [ ]:

```
1 D4={ [1,2]: "One", [3,4]: "Aryan" }
2 print(D4)
```

In [ ]:

```
1 D3={'rollno':1111, 'class':'X', 'sec':'A', 'Clast':'Aman'}
2 print(D3)
3 D3['Clast'] = 'Dhawal'
4 print(D3)
```

# Mutable and Immutable Types

## 1. Immutable Types

1. Immutable types are those that can never change their values in place.
2. Immutable types are:

- a. integers
- b. floating point numbers
- c. booleans
- d. strings
- e. tuples

In [ ]:

```
1 p=5
2 q=p
3 r=5
4 print(id(5))
5 print(id(p))
6 print(id(q))
7 print(id(r))
```

In [ ]:

```
1 p=10
2 r=7
3 q=r
4 print(id(10),id(7))
5 print(id(p))
6 print(id(r))
7 print(id(q))
```

In [ ]:

```
1 a=10
2 print(id(10))
3 print(a)
4 a=20
5 print(id(20))
6 print(a)
```

In [ ]:

```
1 int a=10
2 a=20
```

## 2. Mutable Types

1. The mutable types are those whose values can be changed in place.
2. Mutable types are:
  - a. Lists
  - b. Dictionaries
  - c. Sets

In [ ]:

```
1 print(id(10),id(20),id(30))
2 L=[10,20,30]
3 print(id(L))
4 print(id(40))
5 L[2]=40
6 print(id(L))
```

## Variable Internals

1. Python is an object oriented language.
2. Python calls every entity that stores any values or any type of data as an object.
3. An object is an entity that has certain propertoes and behavior.

***Every Python object has 3 key attributes associated to it:***

1. type()
2. value
3. id()

In [ ]:

```
1 a=10
2 print(type(a))
3 f=3.14
4 print(type(f))
```

In [ ]:

```
1 a=20
2 print(a)
```

In [ ]:

```
1 b=200
2 print(id(200))
3 print(id(b))
```

## Operators

**Unary Operator:**

- unary +
- unary -
- negation (not)

In [ ]:

```
1 + & + = +  
2 + & - = -  
3 - & + = -  
4 - & - = +
```

### ***Example of Unary + and Unary -***

In [ ]:

```
1 a=10  
2 print(+a)  
3 a=-20  
4 print(+a)  
5 a=100  
6 print(-a)  
7 a=-200  
8 print(-a)
```

### **Example of 'not' operator**

In [ ]:

```
1 a=True  
2 print(not a)  
3 b=False  
4 print(not b)
```

In [ ]:

```
1 a=1  
2 print(not a)  
3 a=0  
4 print(not a)
```

In [ ]:

```
1 a=12  
2 print(not a)  
3 a=-12  
4 print(not a)  
5 a=0  
6 print(not a)  
7 a=0.04  
8 print(not a)
```

## **Binary Operators**

### **1. Arithmetic Operator**

In [ ]:

```
1 a,b=10,20
2 print(a+b)
3 print(a-b)
4 print(a/b)
5 print(a*b)
6 print(100/2)
```

### // Floor Division Operator

In [ ]:

```
1 a,b=12,2
2 print(a/b)
3 print(a//b)
```

In [ ]:

```
1 a,b=12.5,4
2 print(a/b)
3 print(a//b)
```

In [ ]:

```
1 a,b=0,2
2 print(a/b)
3 print(a//b)
```

In [ ]:

```
1 Exponent **
```

In [ ]:

```
1 a=10.5
2 b=2.5
3 print(a**b)
```

## Relational Operator

Relational refers to the relationships that values can have with one another, and to check this relationships we will use Relational operator.

In [ ]:

```
1 < > <= >= == !=
```

In [ ]:

```
1 a,b=10,10
2 print(a<b)
3 print(a>b)
4 print(a<=b)
5 print(a>=b)
6 print(a==b)
7 print(a!=b)
```

***Relational Operators work on following principles***

## 1. For numeric types:

- values are compared after removing trailing zeros after decimal point from a floating point number.
- For example:
  - 4 and 4.0 will be treated as equal (after removing trailing zeros from 4.0, it becomes equal to 4 only)

In [ ]:

```
1 a,b=4,4.0
2 print(a==b)
3 a,b=4,4.000000
4 print(a==b)
5 a,b=4,4.1
6 print(a==b)
7 a,b=4,4.01
8 print(a==b)
9 a,b=4,5.000000
10 print(a==b)
```

## 2. For List and Tuples:

- Two lists and Tuples are equal if they have same elements in the same order.

In [ ]:

```
1 L1=[1,2,4,3]
2 L2=[1,2,3,4]
3 print(L1==L2)
```

In [ ]:

```
1 t1=(1,2,4,3)
2 t2=(1,2,3,4)
3 print(t1==t2)
```

## 3. Boolean:

- Boolean True is equivalent to 1(numeric one) and Boolean False to 0(numeric zero) for comparison purposes.

In [ ]:

```

1 A=True
2 B=False
3 print(A==B)

```

In [ ]:

```

1 a=True
2 print(a==1)
3 print(a==0)

```

#### 4. Strings:

- Strings are compared on the basis of Lexicographical ordering(ordering in dictionary).
- Capital letters are considered lesser than small letters.
- ASCII concept: -A=65,B=66,.....Z=90 -a=97,b=98,.....z=122 -0=48,1=49,.....9=57
- To display ordinal/ASCII number of any letter, we will use ord() function
- To display character of any ASCII number , we will use chr() function.

In [6]:

```

1 ASCII=input("Enter a character:- ")
2 print(ord(ASCII))

```

Enter a character:- 9  
57

In [9]:

```

1 Letter=int(input("Enter ASCII:- "))
2 print(chr(Letter))

```

Enter ASCII:- 98  
b

In [40]:

```

1 a,b,p=3,13,3.0
2 print(a<b)
3 print(a<p)
4 print(a==True)

```

True  
False  
False

In [41]:

```

1 c,d,e='n','g','N'
2 print(c<d)
3 print(c==e)

```

False  
False

In [46]:

```
1 f,g,h,j,k='god','God','God','God','Godhouse'
2 print(f<h)    #'god' < 'God'
3 print(f==h)   #'god' == 'God'
4 print(g==j)   #'God' == 'God'
5 print(g<k)
6 print(h<k)
```

False

False

True

True

True

## 3 Identity Operator

1. The identity operators are used to check if both the operands reference the same object memory.
2. There are two identity operators in Python:

a. is

b. is not

3. The identity operators compare the memory locations of two objects and return True or False accordingly.

In [11]:

```
1 a=10
2 b=10
3 print(id(10))
4 print(id(a),id(b))
5 print(a == b)
6 print(a is b)
7 print(a is not b)
```

1999400288

1999400288 1999400288

True

True

False

## Equality(==) and identity(is/is not) - Important Relation

In [13]:

```
1 a=10
2 b=int(input("Enter a value of b:- "))
3 print(a,b,id(a),id(b))
4 print(a==b)
5 print(a is b)
```

Enter a value of b:- 10  
10 10 1999400288 1999400288  
True  
True

**Python creates two different objects even though both store the same value, These are:**

1. input of strings from the console.
2. writing integers literals with many digits(very big integers)
3. writing floating point and complex literals.

In [15]:

```
1 a='aman'
2 b=input("Enter value of b:- ")
3 print(a,b,id(a),id(b))
4 print(a==b)
5 print(a is b)
```

Enter value of b:- aman  
aman aman 2198022329768 2198022411464  
True  
False

In [16]:

```
1 a=1.5
2 b=float(input("Enter value of b:- "))
3 print(a,b,id(a),id(b))
4 print(a==b)
5 print(a is b)
```

Enter value of b:- 1.5  
1.5 1.5 2198012189456 2198012189600  
True  
False

In [26]:

```
1 a=999
2 b=int(input("Enter value of b:- "))
3 print(a,b,id(a),id(b))
4 print(a==b)
5 print(a is b)
```

Enter value of b:- 999  
999 999 2198022904848 2198022902608  
True  
False

## 4. Logical Operators

1. Logical operators can combine results of any two/more expressions into single result.
2. There are 3 logical operators are :

a. and

b. or

c. not

### Logical Operators with Relational expressions (as operands)

## OR

In [ ]:

```
1 X      or      Y = Result
2 False or False = False
3 False or True  = True
4 True  or False = True
5 True  or True  = True
```

In [42]:

```
1 x,y=100<20,1000<200
2 print(x , " ", y , " ", x or y )
3 x,y=100<20,100<200
4 print(x , " ", y , " ", x or y )
5 x,y=10<20,1000<200
6 print(x , " ", y , " ", x or y )
7 x,y=10<20,100<200
8 print(x , " ", y , " ", x or y )
```

```
False  False  False
False  True   True
True   False  True
True   True   True
```

## AND

In [ ]:

```
1 X      and      Y = Result
2 False and False = False
3 False and True  = False
4 True  and False = False
5 True  and True  = True
```

In [41]:

```

1 x,y=100<20,1000<200
2 print(x," ",y," ",x and y )
3 x,y=100<20,100<200
4 print(x," ",y," ",x and y )
5 x,y=10<20,1000<200
6 print(x," ",y," ",x and y )
7 x,y=10<20,100<200
8 print(x," ",y," ",x and y )

```

```

False False False
False True False
True False False
True True True

```

## NOT

In [53]:

```

1 a=True
2 print(not a)
3 a=False
4 print(not a)

```

```

False
True

```

In [27]:

```

1 age=int(input("Enter age value:- "))
2 result1 = age>=5
3 result2 = age<=18
4 print(result1 and result2)

```

```

Enter age value:- 3
False

```

In [35]:

```

1 city = input("Enter value of City:- ")
2 result1 = city=='abc'
3 result2 = city=='xyz'
4 result3 = city=='123'
5 print(result1 or result2 or result3)

```

```

Enter value of City:- aman
False

```

## Truth Value Testing

1. Python associates with every value type, some truth value,
2. where Truth Value - true represents some meaning(value) of data/value/literals
3. whereas truth value - false represents zero-ness or emptiness of data/value/literals.

**Values with truth value as false**

In [ ]:

```

1 None
2 False(boolean value false)
3 zero of any numeric type example-(0,0.0,0j)
4 any empty sequence example-'', '', '','', [], ()
5 any empty mapping example- {}

```

**Values with truth value as true**

All other values are considered as true.

**Logical Operators with numbers/strings/lists as operands****OR**

In an expression X or Y , if first operand(expression X) has falseTV then return second operand Y as result, otherwise return X

In [13]:

```

1 result = 0 or 20
2 print(result)
3 result = 0 or 0.0
4 print(result)
5 result = '' or 'aman'
6 print(result)
7 result = "aman" or 'aryan'
8 print(result)
9 result = [] or [1,2,3]
10 print(result)
11 result = [1,2,3] or []
12 print(result)
13 result = None or [0] or [110,20]
14 print(result)

```

```

20
0.0
aman
aman
[1, 2, 3]
[1, 2, 3]
[0]

```

**AND**

In an expression X and Y if first operand(expression X) has falseTV then return first operand X as result, otherwise return Y

In [23]:

```
1 result = 0 and 20
2 print(result)
3 result = 0.10 and 0
4 print(result)
5 result = 'aman' and ''
6 print(result)
7 result = "aman" and 'aryan'
8 print(result)
9 result = [] and [1,2,3]
10 print(result)
11 result = [1,2,3] and []
12 print(result)
13 result = 0 and [] and [110,20]
14 print(result)
```

0  
0

aryan  
[]  
[]  
0

In [31]:

```
1 20 > 10 or "a" + 10 < 5
```

Out[31]:

True

In [34]:

```
1 10 > 20 and 20 + 10/0
```

Out[34]:

False

## Chained Comparison Operators

In [3]:

```
1 10<20 and 20<30
```

Out[3]:

True

In [4]:

```
1 10<20<30
```

Out[4]:

True

In [5]:

```
1 100 <= 500 and 500 >= 400
```

Out[5]:

True

In [6]:

```
1 100<=500>=400
```

Out[6]:

True

## Bitwise Operators

**Bitwise Operators are :**

1. & (bitwise and)
2. | (bitwise or)
3. ^ (bitwise XOR)
4. ~ (bitwise complement)

**Python built-in function bin() function:**

- It can convert given number into binary form.
- Format :
  - bin(number)

In [8]:

```
1 print(bin(10))
2 print(bin(11))
```

0b1010

0b1011

## The AND operator &

When its operands are numbers, the & operation performs the bitwise AND function on each parallel pair of its bits in each operand.

Op1 & Op2 = Result  
0 & 0 = 0 0 & 1 = 0 1 & 0 = 0 1 & 1 = 1

In [11]:

```

1 a,b=13,12
2 print("Binary of 13:- ",bin(13))
3 print("Binary of 12:- ",bin(12))    # 1100
4 result = 13 & 12
5 print("Binary of result:- ",bin(result)," Number:- ",result)

```

Binary of 13:- 0b1101

Binary of 12:- 0b1100

Binary of result:- 0b1100 Number:- 12

## The inclusive OR operator

1. When both of its operands are numbers, the | operator performs the inclusive OR operation.
2. Inclusive OR means that if either of the two bits is 1, the result is 1.

In [ ]:

| Op1 |  | Op2 | = | Result |
|-----|--|-----|---|--------|
| 0   |  | 0   | = | 0      |
| 0   |  | 1   | = | 1      |
| 1   |  | 0   | = | 1      |
| 1   |  | 1   | = | 1      |

In [15]:

```

1 a,b=13,12
2 print("Binary of 13:- ",bin(13))
3 print("Binary of 12:- ",bin(12))    # 1100
4 result = 13 | 12
5 print("Binary of result:- ",bin(result)," Number:- ",result)

```

Binary of 13:- 0b1101

Binary of 12:- 0b1100

Binary of result:- 0b1101 Number:- 13

## The eXclusive OR(XOR) operator ^

In [ ]:

| Op1 | ^ | Op2 | = | Result |
|-----|---|-----|---|--------|
| 0   | ^ | 0   | = | 0      |
| 0   | ^ | 1   | = | 1      |
| 1   | ^ | 0   | = | 1      |
| 1   | ^ | 1   | = | 0      |

In [17]:

```

1 a,b=13,12
2 print("Binary of 13:- ",bin(13))
3 print("Binary of 12:- ",bin(12))    # 1100
4 result = 13 ^ 12
5 print("Binary of result:- ",bin(result)," Number:- ",result)

```

Binary of 13:- 0b1101

Binary of 12:- 0b1100

Binary of result:- 0b1 Number:- 1

## The Complement Operator ~

In [ ]:

```

1 Skip for the chapter of Boolean Alg

```

## Combined Class Topics

## Operator Precedence

Operator precedence represents the priority of operator in TOP to Bottom approach.

## Operator Associativity

Operator associativity guide processing of any expression whereas expression contains same priority of operators, It usually follows Left to Right or Right to Left approach.

**\*\* follows R2L Associativity**

## Some Examples are

In [39]:

```

1 print(7*8/5//2)
2 result = (((7*8)/5)//2)
3 print(result)
4 result = 7*(8/(5//2))
5 print(result)

```

5.0

5.0

28.0

In [42]:

```
1 result = (3 ** 3) ** 2
2 print(result)
```

729

## Expressions

1. An expressions in Python is any valid combination of operators and atoms(identifiers,literals,strings,list,tuples and dictionaries).
2. The types of operators and operands used in an expressions determine the expression type.

***The expressions in Python can be of any type according to used operators:***

In [ ]:

```
1 1. Arithmetic Exp
2 2. Relational Exp
3 3. Logical Exp
4 4. String Exp(sequence Exp)
5 5. Mixed(complex) expression
```

### **1. Arithmetic Exp:**

- 10+20 , a+b , a/bb

### **2. Relational Exp:**

- 10<20 , a>20 , a>=b

### **3. Logical Exp:**

- a and b , 10 or 20 and 30

### **4. String Exp:**

- "aman" + "sharma" , "aman" \* 3

### **5. Mixed Expr:**

- 10+20>30-20 and 10\*2\*1

## Evaluating Expressions

## Evaluating Arithmetic Expression

In [47]:

```
1 A = 5 + 2 / 5.0
2 print(A)
```

5.4

## Evaluating Relational Expression

In [5]:

```
1 result = 10<20 == 20>=20 != 30>20
2 print(10<20==21)
```

False

## Evaluating Logical Expression

In [ ]:

```
1
```

## Evaluating String Expression

In [ ]:

```
1 s='aman '
2 s="aman"
```

Use of + operator in between two strings:

**+ operator basically perform concatenation operation between strings**

In [26]:

```

1 s1='Python'
2 s2='Language'
3 print(s1 + s2)
4 print(s1 + 10)

```

PythonLanguage

```

-----
TypeError                                Traceback (most recent call last)
<ipython-input-26-1e394c841ad7> in <module>()
      2 s2='Language'
      3 print(s1 + s2)
----> 4 print(s1 + 10)

```

**TypeError:** must be str, not int

### Use of \* operator

**\* operator replicate given string into specific number of times.**

In [19]:

```

1 s1="Python"
2 print(s1 * 5)
3 print(5 * s1)
4 print(s1 * '5')

```

PythonPythonPythonPythonPython  
 PythonPythonPythonPythonPython

```

-----
TypeError                                Traceback (most recent call last)
<ipython-input-19-e48b13d47f41> in <module>()
      2 print(s1 * 5)
      3 print(5 * s1)
----> 4 print(s1 * 1+2)

```

**TypeError:** must be str, not int

## Evaluating Mixed Expression

## Type Casting/Conversion

Type Casting is the process of coverting/casting one type of value into another type of value.

### Their two types of type casting present:

1. Implicit Type Casting
2. Explicit Type Casting

## 1. Implicit Type Casting

1. An implicit type conversion is a conversion performed by the compiler/interpreter without programmer's intervention.
2. This type of conversion take place in only those expression which are intermixed(differet type values are present).
3. In a mixed arithmetic expression, Python covert's all operands UP to the tupe of the largest operand - this is called TYPE PROMOTION:
  - a. If either argument(operand) is a complex number the other is covered into complex.
  - b. Otherwise, if either operand is a floating point number the other is covered to floating point
  - c. No conversion if both operands are integers.

In [22]:

```
1 a,b=3,6
2 print(b*a)
```

18

In [23]:

```
1 a,b=3,6.0
2 print(b*a)
```

18.0

In [24]:

```
1 a,b=3,6
2 print(b/a)
3 print(b//a)
```

2.0

2

In [25]:

```
1 a,b=3,6.6
2 print(b/a)
3 print(b//a)
```

2.1999999999999997

2.0

## 2. Explicit Type Casting

- 1.An explicit type casting is user-defined conversion that forces an expression to be of specific type. 2.The explicit type conversion is also known as Type Casting.

**How to perform Type Casting - Format:**

<datatype>(expression)

In [ ]:

```
1 All different types of Python data conversion Functions:
2   - int()
3   - float()
4   - str()
5   - complex()
6   - bool()
```

In [17]:

```
1 print(int(3.14))
2 print(float(3))
3 a=100
4 a=str(a)
5 print(a,type(a))
6 print(complex(1))
7 print(complex(1,2))
8 print(bool(1))
9 print(bool(0.0))
10 print(bool(' '))
11 print(bool([]))
```

```
3
3.0
100 <class 'str'>
(1+0j)
(1+2j)
True
False
False
False
```

## Introduction to Python Standard Library Modules

Module - It is a collection of function, methods, constants.

math random statistics

### How to use/import/include module

In [ ]:

```
1 import <module/package/library name> as <alias name>
```

In [18]:

```
1 import math
```

In [19]:

```
1 print(math.pi)
```

3.141592653589793

In [23]:

```
1 print(10**3.2)
```

1584.893192461114

In [24]:

```
1 base=float(input("Enter Base Value:- "))
2 power=float(input("Enter Power Value:- "))
3 result = math.pow(base,power)
4 print(result)
```

Enter Base Value:- 2

Enter Power Value:- 5

32.0

In [25]:

```
1 print(math.sqrt(121))
```

11.0

In [26]:

```
1 print(11/2)
2 print(11%2)
```

5.5

1

In [30]:

```
1 print(int(math.fmod(11,2)))
```

1

In [32]:

```
1 print(math.factorial(10))
```

3628800

## Using the statistics module

This modules provide statistics functions such as mean(),median(),mode(),etc.

In [33]:

```
1 import statistics
```

In [36]:

```
1 l=[5,6,7,5,6,5,5,9,11,12,23,5]
2 print(statistics.mean(l))
3 print(statistics.mode(l))
4 print(statistics.median(l))
```

8.25

5

6.0

## Using random Module

In [ ]:

```
1 Python has a module random that provides random number generators.
```

**How to use random module and their methods:**

```
import random
```

In [1]:

```
1 import random
```

**1st method of random module**

In [ ]:

```
1 random.random():
2     - it returns a random floating point number N in the range of [0.0,1.0]
3     - Where 0.0<=N<1.0
```

In [34]:

```
1 print(round(random.random(),2))
```

0.95

**2nd method of random module**

In [ ]:

```
1 random.randint(a,b):
2     - It returns a random integer N in the range of(a,b)
3     - a<=N<=b
```

In [162]:

```
1 print(random.randint(1,10))
```

7

### 3rd method of random module

In [ ]:

```
1 random.randrange():  
2     - It returns random numbers from range start.... stop with step value.  
3     - (start,stop,step)
```

In [173]:

```
1 print(random.randrange(1,5,1))
```

1

In [248]:

```
1 for i in range(1,11):  
2     result = round(random.random()*5)  
3     print(result)
```

3

1

4

4

4

4

0

5

1

4

In [256]:

```
1 for i in range(1,11):  
2     result = random.randint(2,10)+2  
3     print(result)
```

10

11

7

5

10

4

4

4

8

12

## Debugging

Debugging refers to the process of locating the place of error, cause of error, and correcting the code accordingly.

## Error/Bug

An error, sometimes called 'a bug', is anything in the code that prevents a program from compiling and running

correctly.

### **Three Types of Errors are:**

1. Compile Time Error
  - a. Syntax Error
  - b. Semantics Error
2. Run Time Error
3. Logical Error

#### **1. Compile Time Error:**

- Errors that occur during compile time are compile time error.
- During compilation source code is checked for whether it follows the programming language's rules or not
- Two types of compile time errors:
  - a. Syntax
  - b. Semantics

### **What is Syntax?**

Syntax refers to formal rules governing the construction of valid statements in a language.

## **Syntax Error**

Syntax errors occur when rules of a programming language are misused.

In [ ]:

```
1 x <= 10+20
2 Input()
3 true
```

## **Semantics Errors**

Semantics errors occurs when statements are not meaningful.

### **What is Semantics meaning?**

Semantics refers to the set of rules which given the meaning of a statement.

In [45]:

```

1 A,B,C = 10,20,30
2 Z=A+B+c
3 A+B+C = Z

```

File "<ipython-input-45-659eb265bbf0>", line 2  
 A+B+C = Z  
       ^

**SyntaxError:** can't assign to operator

## 2. Logical Error

Logical errors are those where we can't find any syntactical as well as semantical errors but even after this my program will be showing unexpected output due to logical bug.

In [47]:

```

1 a,b=10,20
2 print("Addition of two number",a,"+",b,"= ",a*b)

```

Addition of two number 10 + 20 = 200

## 3. Run Time Error

In [ ]:

```

1 1. This error occur during run time of the program
2 2. This type of error is harder to find.

```

In [48]:

```

1 a=int(input("Enter value"))
2 print(a)

```

Enter valuesdsd

```

-----
ValueError                                Traceback (most recent call last)
<ipython-input-48-4412cdcef6ed> in <module>()
----> 1 a=int(input("Enter value"))
      2 print(a)

```

**ValueError:** invalid literal for int() with base 10: 'sdsd'

## Exceptions

Exception is an irregular unexpected situation occurring during execution on which programmer has no control

In [49]:

```
1 print(10/0)
```

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
<ipython-input-49-fe01563e1bc6> in <module>()  
----> 1 print(10/0)
```

**ZeroDivisionError**: division by zero

In [52]:

```
1 L=[10,20,30]  
2 print(L[2])
```

30

In [57]:

```
1 import math  
2 math.asine()
```

```
-----  
AttributeError                                Traceback (most recent call last)  
<ipython-input-57-8ee092cfb9ff> in <module>()  
      1 import math  
----> 2 math.asine()
```

**AttributeError**: module 'math' has no attribute 'asine'

## End of Chapter

## Next Chapter - Flow of Control

True For false And or none import Pass

In [ ]:

```
1 0o127  
2 00122
```

In [ ]:

```
1 1. Characters written in '' pair  
2 2. Characters written in ''' ''' pair
```

In [ ]:

```
1 1. Text written in '' pair  
2 2. Text written in " " pair where each line end by \
```

In [ ]:

```
1 10 + 20 <= 20 > 30
```

In [ ]:

```
1 / // * <= and or **
```

In [1]:

```
1 a,b,c = 2,3,4
2 a,b,c = a*a , a*b , a*c
3 print(a,b,c)
```

4 6 8

In [2]:

```
1 x,y = 20,60
2 y,x,y = x, y-10 , x+10      #20 , 50 , 30
3 print(x,y)
```

50 30

In [3]:

```
1 print(type(0))
2 print(type(int(0)))
3 print(type(int('0')))
4 print(type(float(0)))
```

```
<class 'int'>
<class 'int'>
<class 'int'>
<class 'float'>
```

In [ ]:

```
1 [] = list
2 () = int,float,tuple
3 {} = Sets and Dictionary
```

In [ ]:

```
1 int
2 float
3 string
4 complex
5 list
6 tuple
7 sets
8 dictionary
```

## Programs

Q.1 Write a program that inputs a string and then prints it equal to number of times its length, e.g. Enter string : "eka" Result ekaekaeka

Q.2 Write a program to generate 6 random numbers and then print their mean, median and mode.

Q.3 Write a program to calculate amount payable after simple interest.

Q.4 Write a program to calculate amount payable after Compound interest.

Q.5 Write a program to generate 6 random secure OTP between 10000 to 99999.